SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I Ashwanth SP/192126002 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: ARTIFICIAL INTELLIGENCE

COURSE CODE: CSA-1709

DONE BY: ASHWANTH SP (192525014)

ANALYTICAL PROBLEM SOLVING

QUESTION 1: WATER JUG PROBLEM

The objective is to obtain exactly 2 gallons in the 4-gallon jug using a 4-gallon jug and a 3-gallon jug. The jugs have no measuring marks, and the permitted operations are filling a jug from the pump, emptying water onto the ground, and pouring water from one jug into the other. Represent each state as (x, y), where x is the amount in the 4-gallon jug and y is the amount in the 3-gallon jug.

Initially, both jugs are empty, so the state is (0, 0). Step 1: Fill the 3-gallon jug, producing (0, 3). Step 2: Pour the 3-gallon jug into the 4-gallon jug, producing (3, 0). Step 3: Fill the 3-gallon jug again, producing (3, 3). Step 4: Pour from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full. Only 1 gallon fits, so the resulting state is (4, 2). Step 5: Empty the 4-gallon jug, producing (0, 2). Step 6: Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug, producing (2, 0). The 4-gallon jug now contains exactly 2 gallons.

Step	Operation	State (4-gal, 3-gal)	Result
0	Start	(0, 0)	Both empty
1	Fill 3-gallon jug	(0, 3)	3-gallon jug full
2	Pour 3-gallon into 4-gallon	(3, 0)	4-gallon jug has 3
3	Fill 3-gallon jug	(3, 3)	Both contain 3
4	Pour into 4-gallon until full	(4, 2)	2 gallons remain in 3-gallon jug
5	Empty 4-gallon jug	(0, 2)	2 gallons remain in 3-gallon jug
6	Pour 3-gallon into 4-gallon	(2, 0)	Goal achieved

Final answer: The state (2, 0) means the 4-gallon jug contains exactly 2 gallons and the 3-gallon jug is empty.

QUESTION 2: MARS ROVER AS AN INTELLIGENT AGENT

A Mars Rover is an autonomous robotic agent that senses its surroundings, moves across the Martian surface, examines rocks and soil, and collects or analyzes samples. Its decisions must account for limited communication with Earth, uncertain terrain, energy limits, scientific priorities and equipment safety.

(i) Percepts of the agent

The rover's percepts include camera images, terrain and obstacle measurements, wheel slip, tilt and orientation, distance measurements, temperature, battery and power status, instrument readings, and the chemical or mineral composition detected in rocks and soil. These percepts allow the rover to estimate its location, recognize hazards, identify scientifically interesting targets, and monitor its own condition.

(ii) Operating environment

The environment is partially observable because sensors cannot reveal every detail of the terrain or the interior composition of a sample. It is sequential because each movement affects future positions, energy use and available scientific opportunities. It is dynamic because environmental and rover conditions can change. It is continuous in physical space and time, and it is uncertain because terrain interactions and scientific measurements may not be perfectly predictable. The rover is also a multi-stage autonomous system with communication delays, so it cannot rely on immediate human instructions for every action.

(iii) Actions the agent can take

The rover can move forward or backward, turn, stop, choose a route, capture images, focus or reposition cameras, use a robotic arm, drill or abrade a rock when equipped, collect or place a sample, run onboard scientific instruments, transmit data to Earth, conserve power, and enter a safe mode when a fault or hazard is detected. The exact actions depend on the rover's hardware and mission commands.

(iv) Performance evaluation

Performance can be evaluated by the quality and quantity of useful scientific data, the number and relevance of samples analyzed, successful completion of mission objectives, accuracy of navigation, distance traveled without damage, energy efficiency, communication and data-transfer success, equipment reliability, and the ability to avoid hazards. A good performance measure rewards meaningful scientific discoveries and safe mission completion rather than movement alone.

(v) Most suitable agent architecture

A hybrid architecture combining a model-based, goal-based and utility-based agent is most suitable. A model-based component maintains an internal map and a record of rover state because the environment is only partially observable. A goal-based component plans actions toward mission objectives, such as reaching a target or analyzing a sample. A utility-based component compares alternatives using scientific value, travel risk, energy consumption, time and hardware safety. A learning component can further improve terrain classification and target selection from experience. This combination is suitable because the rover must plan ahead, reason with incomplete information, and balance scientific benefit against risk and resource limits.

QUESTION 3: 8-QUEENS PROBLEM AND SEARCH SPACE

The 8-Queens problem requires placing eight queens on an 8 × 8 chessboard so that no queen attacks another. Since a queen attacks along its row, column and diagonals, a valid configuration must have no two queens in the same row, column or diagonal. A convenient representation places exactly one queen in each column and stores the row of the queen in that column. A state can therefore be represented as [r1, r2, r3, r4, r5, r6, r7, r8], where each ri is a row number from 1 to 8.

Problem formulation: The initial state is an empty board. The state space consists of partial or complete placements of queens. An action places a queen in a legal row of the next column. The goal test succeeds when all eight queens are placed and no pair shares a row or diagonal. With the one-queen-per-column representation, column conflicts are eliminated by design. A diagonal conflict exists between queens in columns i and j if $|ri - rj| = |i - j|$. The path cost can be treated as uniform per placement, or the problem can be formulated as a constraint-satisfaction problem without a path-cost objective.

Search approach: Backtracking Search places a queen in the next column, checks whether it conflicts with any earlier queen, and proceeds only when the placement is safe. If no row is legal in a column, it removes the previous queen and tries another row. This avoids exploring many invalid complete boards. Forward checking can further improve the search by removing rows that have become impossible in future columns.

One valid solution, with the row listed for columns 1 through 8, is [1, 5, 8, 6, 3, 7, 2, 4]. Each column contains one queen, all row values are different, and for every pair of columns i and j the diagonal condition $|ri - rj| \neq |i - j|$ holds. Thus no queen attacks another.

Q	·	·	·	·	·	·	·
·	·	·	·	·	·	Q	·
·	·	·	·	Q	·	·	·
·	·	·	·	·	·	·	Q
·	Q	·	·	·	·	·	·
·	·	·	Q	·	·	·	·
·	·	·	·	·	Q	·	·
·	·	Q	·	·	·	·	·

The figure in the assessment illustrates a configuration that is not a solution because the queen in the first column shares a diagonal with the queen in the last column. The diagonal test above detects this type of conflict and rejects the state.

QUESTION 4: OLA CAB BOOKING AS A PROBLEM-SOLVING AGENT

A goal-based problem-solving agent is suitable for an OLA booking task because the customer has a destination goal and must choose a sequence of actions to reach it. The agent gathers the pickup and destination locations, identifies available cab categories such as Mini, Micro, Sedan, Shared or Prime,

compares estimated fares and arrival times, and selects a feasible option according to the customer's preferences. If the customer prioritizes a combination of price, comfort and waiting time, a utility-based decision layer can rank the feasible options.

Problem formulation: The initial state contains the customer's current location and booking preferences. The goal state is a confirmed cab booking for the selected destination and category. Actions include entering locations, selecting a cab type, requesting fare estimates, confirming a booking, or changing the choice. Constraints include cab availability, fare budget, pickup time, capacity and the customer's comfort preference.

Pseudocode

```
BEGIN
  INPUT pickup_location, destination, preferences, budget
  IF pickup_location or destination is missing THEN
      ASK customer to provide the missing location
  ENDIF
  cab_options ← GET_AVAILABLE_CABS(pickup_location, destination)
  feasible_options ← empty list
  FOR each cab in cab_options DO
      estimate ← GET_FARE_AND_ETA(cab, pickup_location, destination)
      IF cab satisfies preferences AND estimate.fare ≤ budget THEN
          ADD (cab, estimate) TO feasible_options
      ENDIF
  ENDFOR
  IF feasible_options is empty THEN
      SHOW alternatives and ask customer to change preferences or budget
  ELSE
      chosen_cab ← SELECT_BEST_OPTION(feasible_options, preferences)
      SHOW chosen_cab, estimated fare, and ETA
      IF customer confirms THEN
          booking ← CONFIRM_BOOKING(chosen_cab)
          DISPLAY booking details
      ELSE
          ALLOW customer to select another option
      ENDIF
  ENDIF
END
```

The SELECT_BEST_OPTION function should use the customer's stated priorities rather than assume that one cab category is always best. For example, it may minimize fare when budget is the main concern, or balance fare, waiting time and comfort when the customer values all three.

QUESTION 5: UNIFORM COST SEARCH FOR THE DELIVERY NETWORK

The assessment diagram gives a weighted directed graph with start S and goal G. The edge costs shown are S → A = 1, S → G = 12, A → B = 3, A → C = 1, B → D = 3, C → D = 1, C → G = 2, and D → G = 3. Uniform Cost Search (UCS) uses a priority queue ordered by the cumulative cost g(n) from S, rather than by the number of edges. At each iteration, it removes the frontier node with the lowest cumulative cost and relaxes its outgoing edges.

Step-by-step UCS execution

Step	Node expanded / action	Priority queue after update (node: cumulative cost)
1	Start at S, cost 0; add A and G	A:1, G:12
2	Expand A (cost 1); add B at 4 and C at 2	C:2, B:4, G:12
3	Expand C (cost 2); add D at 3 and improve G to 4	D:3, B:4, G:4
4	Expand D (cost 3); route to G would cost 6, worse than 4	B:4, G:4
5	Expand B (cost 4); route to D would cost 7, worse than 3	G:4
6	Remove G (cost 4); goal reached	Stop

Candidate path costs: S → G costs 12. S → A → B → D → G costs 1 + 3 + 3 + 3 = 10. S → A → C → G costs 1 + 1 + 2 = 4. S → A → C → D → G costs 1 + 1 + 1 + 3 = 6. The minimum is 4, so the optimal route is S → A → C → G.

Final answer: UCS selects S → A → C → G with total travel cost 4. UCS is optimal for this graph because all edge costs are non-negative and the goal is accepted when it is removed from the priority queue at the lowest cumulative cost. This minimizes the combined travel cost represented by the graph.

CONCLUSION

The Water Jug problem demonstrates state-space search through legal operations. The Mars Rover illustrates how an intelligent agent uses percepts, actions and performance measures in a partially observable environment. The 8-Queens problem can be solved efficiently with backtracking and constraint checks. OLA cab booking is naturally modeled as a goal-based agent with preference-

aware selection. Finally, Uniform Cost Search finds the least-cost delivery route $S \rightarrow A \rightarrow C \rightarrow G$ with total cost 4.